

Golden Rule -> retrieve top-k relevant chunks, stuff them into the prompt, then ask the LLM to answer only from that context

1. What A Retriever Does

A retriever wraps a vector store: given a query, it returns the top-k chunks most similar to that query. It is the “search” step of RAG — the bridge between the vector store and the LLM.

2. Retriever Types

Type	How It Works	When To Use
Similarity search (top-k)	Returns the k nearest vectors by similarity	Default, simplest case
MMR (Maximal Marginal Relevance)	Balances relevance with diversity between results	Avoid near-duplicate chunks
Similarity score threshold	Only returns results above a similarity cutoff	Avoid injecting low-relevance noise
Hybrid (keyword + vector)	Combines exact keyword match with semantic search	Queries with specific terms, IDs, or names

3. Building The Chain

The full pipeline chains four pieces together: retriever → prompt template (inserts retrieved context + the question) → LLM → answer. LangChain's RetrievalQA (or an LCEL chain) wires this up in a few lines.

4. Code Example

```
$ retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

$ from langchain.chains import RetrievalQA
$ qa_chain = RetrievalQA.from_chain_type(
$ llm=llm,
$ retriever=retriever,
$ return_source_documents=True,
$ )
$ response = qa_chain.invoke({"query": "What is the claim process?"})
```

5. Prompt Template For Grounded Answers

Always instruct the LLM explicitly to stay inside the retrieved context, and to admit when it can't answer:

```
$ Answer the question using ONLY the context below.
$ If the answer isn't in the context, say you don't know.

$ Context:
$ {context}

$ Question: {question}
```

Golden Rule -> retrieve top-k relevant chunks, stuff them into the prompt, then ask the LLM to answer only from that context

Pitfall

If you forget to instruct the LLM to stick to the retrieved context, it will happily blend in outside knowledge (or hallucinate) even though RAG handed it good context.

Remember

`return_source_documents=True` (or the equivalent in your framework) lets you show users exactly which chunks an answer came from — always surface this for trust.

INTERVIEW PREP

Q: What is “k” in top-k retrieval, and how does changing it affect answers?

A: k is the number of chunks retrieved for a query. Too small risks missing relevant information; too large adds noise, drives up context length and cost, and dilutes relevance.

Q: What problem does MMR retrieval solve that plain similarity search doesn't?

A: Plain top-k similarity can return several near-duplicate chunks; MMR re-ranks results to balance relevance with diversity so the LLM sees a broader spread of information.

Q: Why explicitly instruct the LLM to answer “only from the provided context”?

A: Without that instruction the model may fall back on its own parametric knowledge or blend it with retrieved context, defeating the grounding purpose of RAG and reintroducing hallucination risk.

Q: How do you make a RAG answer traceable to its sources?

A: Return the source documents and metadata alongside the generated answer (e.g. `return_source_documents=True`) and display citations to the user.

Q: When would you reach for hybrid (keyword + vector) retrieval instead of pure vector search?

A: When queries include exact terms like product codes, names, or IDs that semantic similarity alone might miss — keyword search catches the exact term while vector search catches paraphrases.